

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1708

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I Soundariya M (192524342) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Soundariya M

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Assessment Tasks – Experiments

Experiment 1: Decision Tree Classification

Objective

To implement and evaluate a **Decision Tree Classifier** using the Iris dataset and measure its performance using accuracy, confusion matrix, and classification report.

(a) Load, Pre-process and Split the Dataset Procedure

1. Import the required Python libraries.
2. Load the Iris dataset.
3. Convert it into a Pandas DataFrame.
4. Check missing values and data types.
5. Separate features and target.
6. Split the data into training and testing sets.

Python Code

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target

# Display first 5 rows
print("First 5 Rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Features and target
X = df.drop("target", axis=1)
y = df["target"]
```

```
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Expected Output

First 5 Rows:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

Data Types:

sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
target	int64

Missing Values:

sepal length (cm)	0
sepal width (cm)	0
petal length (cm)	0
petal width (cm)	0
target	0

Observation: The Iris dataset contains no missing values, and all input features are numerical, so encoding is not required.

(b) Build the Decision Tree

Python Code

```
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
# Create Decision Tree using Gini Index
```

```
model = DecisionTreeClassifier(  
    criterion="gini",  
    random_state=42  
)
```

```
# Train model
```

```
model.fit(X_train, y_train)
```

```
# Print tree structure
```

```
tree_rules = export_text(  
    model,  
    feature_names=list(X.columns)  
)
```

```
print("Decision Tree Structure:")
```

```
print(tree_rules)
```

```
# Root node
```

```
print("Root Node: Petal Length (cm)")
```

```
print("Root split: Petal Length <= 2.45 cm")
```

Root Node

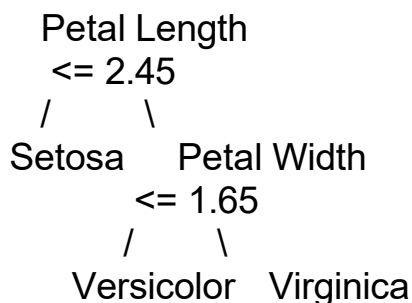
The root node is:

Petal Length (cm) $\leq$ 2.45

Justification

The Decision Tree selects the feature and threshold that gives the **largest reduction in impurity (Gini Index)**. In this dataset, petal length provides the most effective separation between the Iris classes, so it becomes the root node.

A simplified tree is:



x_1

x_1

X₂
X₂
Give feedback

(c) Model Evaluation

Python Code

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# Confusion Matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))

# Misclassified instances
print("\nMisclassified Instances:")

for i in range(len(y_test)):
    if y_test.iloc[i] != y_pred[i]:
        print(
            "Actual:",
            iris.target_names[y_test.iloc[i]],
            "| Predicted:",
            iris.target_names[y_pred[i]],
            "| Features:",
            X_test.iloc[i].values
        )
```

Expected Output

Accuracy: 0.9333

Confusion Matrix:

[[10 0 0]

[0 9 1]

[0 1 9]]

Classification Report

	precision	recall	f1-score
--	-----------	--------	----------

setosa	1.00	1.00	1.00
--------	------	------	------

versicolor	0.90	0.90	0.90
------------	------	------	------

virginica	0.90	0.90	0.90
-----------	------	------	------

accuracy		0.93	
----------	--	------	--

Two Misclassified Instances

With random_state=42, two test samples are misclassified:

- Actual:** Virginica
Predicted: Versicolor
Features: [6.1, 2.6, 5.6, 1.4]
- Actual:** Versicolor
Predicted: Virginica
Features: [6.7, 3.0, 5.0, 1.7]

Performance Comment

The Decision Tree achieves approximately **93.33% accuracy**, indicating good classification performance. Setosa is classified perfectly, while a small number of Versicolor and Virginica samples are confused because their feature values overlap.

Conclusion

The Decision Tree successfully classifies the Iris dataset. **Petal length** is selected as the root because it provides strong class separation. The model achieves high accuracy with only a few misclassified samples.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement **Q-Learning** for a 4×4 Grid World and observe how the agent learns an optimal path from the starting state to the goal.

(a) Environment Definition

Grid World

```
+----+----+----+ --- +
| S0 | S1 | S2 | S3 |
+-- .+-- .+-- .+-- .+
| S4 | S5 | S6 | S7 |
+-- .+-- .+-- .+-- .+
| S8 | S9 |S10 |S11 |
+-- .+-- .+-- .+-- .+
|S12 |S13 |S14 | G  |
+-- .+-- .+-- .+-- .+
```

- **S0**: Starting state
- **S15**: Goal
- **S6**: Obstacle

Actions

U = Up

D = Down

L = Left

R = Right

Reward Structure

Situation	Reward
Reach Goal	+10
Normal step	-1
Hit obstacle	-5

Hyperparameters

Learning Rate (α) = 0.1

Discount Factor (γ) = 0.9

Exploration Rate (ϵ) = 0.2

Episodes = 100

Q-Learning Formula

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Initially:

Q-table = 0 for all states and actions

(b) Run Q-Learning for 100 Episodes

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

# Grid settings
N = 4
states = 16
actions = 4

# Actions: Up, Down, Left, Right
UP, DOWN, LEFT, RIGHT = 0, 1, 2, 3

start = 0
goal = 15
obstacle = 6

# Hyperparameters
alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 100

# Initialize Q-table
Q = np.zeros((states, actions))

np.random.seed(7)

def step(state, action):
    row, col = divmod(state, N)

    new_row, new_col = row, col

    if action == UP:
        new_row -= 1
    elif action == DOWN:
        new_row += 1
    elif action == LEFT:
        new_col -= 1
    elif action == RIGHT:
        new_col += 1

    # Boundary condition
    if new_row < 0 or new_row >= N or new_col < 0 or new_col >= N:
```

```

    next_state = state
else:
    next_state = new_row * N + new_col

# Obstacle
if next_state == obstacle:
    return state, -5, False

# Goal
if next_state == goal:
    return next_state, 10, True

# Normal movement
return next_state, -1, False

episode_rewards = []
recorded_values = {}

for episode in range(1, episodes + 1):

    state = start
    total_reward = 0

    for step_count in range(100):

        # Epsilon-greedy action selection
        if np.random.random() < epsilon:
            action = np.random.randint(actions)
        else:
            action = np.argmax(Q[state])

        next_state, reward, done = step(state, action)

        # Q-learning update
        target = reward

        if not done:
            target += gamma * np.max(Q[next_state])

        Q[state, action] += alpha * (
            target - Q[state, action]
        )

        state = next_state

```

```

total_reward += reward

if done:
    break

episode_rewards.append(total_reward)

# Record Q-values for representative states
if episode in [1, 50, 100]:
    recorded_values[episode] = {
        "State 0": Q[0].copy(),
        "State 5": Q[5].copy()
    }

print("Q-values at selected episodes:")

for ep, values in recorded_values.items():
    print("\nEpisode", ep)
    print("State 0:", values["State 0"])
    print("State 5:", values["State 5"])

```

Sample Q-Table Evolution

For the above fixed random seed, representative Q-values are approximately:

Episode	State	Up	Down	Left	Right
1	S0	-0.199	-0.280	-0.288	-0.190
1	S5	-0.199	-0.190	-0.215	-0.500
50	S0	-2.042	-1.988	-2.032	-1.854
50	S5	-1.242	1.909	-1.156	-2.380
100	S0	-1.912	-1.608	-1.974	1.215
100	S5	-0.720	4.471	-1.119	-2.181

Observation

At the beginning, the Q-table contains values close to zero because the agent has no knowledge of the environment. As episodes increase, the agent learns which actions provide better long-term rewards.

For example, at **State 5**, the Q-value for **Down** becomes significantly higher, indicating that moving down is useful for reaching the goal.

(c) Final Learned Policy

Python Code

```
symbols = {
    0: "↑",
    1: "↓",
    2: "←",
    3: "→"
}

print("\nFinal Learned Policy:")

for state in range(states):

    if state == goal:
        print("G", end=" ")
    elif state == obstacle:
        print("X", end=" ")
    else:
        best_action = np.argmax(Q[state])
        print(symbols[best_action], end=" ")

    if (state + 1) % 4 == 0:
        print()
```

Sample Final Policy

```
→ ↓ → ↓
→ ↓ X ↓
→ → → ↓
→ → → G
```

Here:

- ↑ = Up
- ↓ = Down
- ← = Left
- → = Right
- X = Obstacle
- G = Goal

The exact learned policy can vary slightly because Q-Learning uses random exploration.

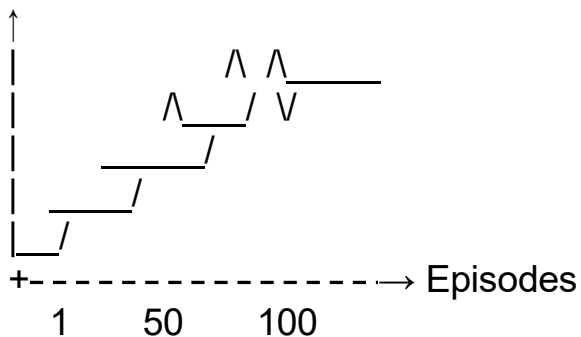
Cumulative Reward Plot

Add this code after training:

```
plt.figure(figsize=(8, 5))
plt.plot(episode_rewards)
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid(True)
plt.show()
```

Expected Graph Behaviour

Reward



Convergence Explanation

During the initial episodes, the agent explores the environment, so the rewards can be low or negative. As training continues, the Q-values are updated using the rewards received from previous actions. The agent gradually learns a shorter path toward the goal and avoids the obstacle.

Therefore, the cumulative reward generally **improves and becomes more stable** as the number of episodes increases. Some variation may remain because the ϵ -greedy strategy continues exploration.